

```

#pragma once

// Debug Macros, for printing to the Serial Monitor
// only when 'DEBUG_MODE' is defined

#include <stdint.h>

#define MSG_TERMINATOR '^'

#define ReqActBox "RAB"
#define UpdBoxes "UPD"
#define ReqInvCount "IVC"
#define ReqInvWin "IVW"

#define INVENTORY_FILE "/inventory.csv"

#define PROTO_VERSION 0x01

// Debug defines, made such that when debug mode is off (and the system is in
// production mode) various types of processes strictly for debugging purposes
// are turned off such that the system can focus on main tasks.
#define DEBUG_MODE

#ifndef DEBUG_MODE

    #ifndef NODEMCU_FILE
        #define d_SerialPrint(x) Serial.print(x)
        #define d_SerialPrintln(x) Serial.println(x)
    #else
        #define d_SerialPrint(x) Serial.print(F(x))
        #define d_SerialPrintln(x) Serial.println(F(x))
    #endif
    #define d_SerialBegin(x) Serial.begin(x)
    #define d_SerialPrintV(x) Serial.print(x) // Print Value
    #define d_SerialPrintlnV(x) Serial.println(x) // Print Value
    #define d_SerialPrintf(...) Serial.printf(__VA_ARGS__)
    #define d_SerialPrintGroup(...) Serial.print(__VA_ARGS__)
    #define d_SerialPrintGroupln(...) Serial.println(__VA_ARGS__)
    #define d_delay(x) delay(x)

#else

    #define d_SerialBegin(x)
    #define d_SerialPrint(x)
    #define d_SerialPrintV(x)
    #define d_SerialPrintln(x)
    #define d_SerialPrintlnV(x)
    #define d_SerialPrintf(...)
    #define d_SerialPrintGroup(...)

```

```

#define d_SerialPrintGroupLn(...)
#define d_delay(x)

#endif

// Eponymous.
enum FoodCondition : uint8_t
{
    FRESH = 0,
    GOING_BAD = 1,
    EXPIRED = 2,
    UNKNOWN = 3,
};

// Struct type to easily pack value types.
struct ThresholdValues
{
    uint16_t alc_bad;
    uint16_t alc_exp;
    uint16_t ch3_bad;
    uint16_t ch3_exp;
};

enum ProfileID : uint8_t
{
    PROFILE_UNKNOWN = 0,
    FRUIT_FAST = 1,
    FRUIT_MEDIUM = 2,
    FRUIT_SLOW = 3,
    EXPERIMENTAL = 4,
    TIME_ONLY = 5
};

// To turn a sensor 'off', simply set the threshold value to 9999.
const ThresholdValues profile_table[] = {
    {9999, 9999, 9999, 9999}, // UNKNOWN
    {180, 320, 9999, 9999},   // FRUIT_FAST
    {220, 380, 9999, 9999},   // FRUIT_MEDIUM
    {260, 440, 9999, 9999},   // FRUIT_SLOW
    {220, 380, 500, 700},     // EXPERIMENTAL
    {9999, 9999, 9999, 9999}  // TIME_ONLY
};

// Wifi Password Pair Type
typedef struct
{
    char ssid[20];
    char pass[20];
} wifi_pass_t;

```

```

// Boxes Type
struct box_t
{
    char index[3];
    volatile FoodCondition State;
    volatile ProfileID p_id;
    uint64_t address;
};

typedef enum
{
    BoxInfo,
    ActiveBoxes,
    UpdateBoxes
} Requests;

struct Packet
{
    uint8_t header;
    uint8_t payload;
    uint8_t crc;
};

enum PacketType : uint8_t
{
    PKT_REQUEST = 0,
    PKT_RESPONSE = 1,
    PKT_CONFIG = 2
};

inline constexpr uint64_t CalcAddress(const char *index)
{
    return (0xF0F0F0F000LL | (uint64_t)index[0]) | ((uint64_t)index[1] << 8);
}

inline uint8_t MakeHeader(uint8_t type, uint8_t sequence)
{
    return ((PROTO_VERSION & 0x01) << 7) | ((type & 0x03) << 5) | (sequence & 0x0F);
}

inline uint8_t MakePayload(FoodCondition condition, uint8_t additional_data = 0)
{
    return ((condition & 0x03) << 6) | (additional_data & 0x3F);
}

inline uint8_t ComputeCRC(uint8_t header, uint8_t payload)
{
    return header ^ payload;
}

```

```

}

// Packs data into a valid 3-byte packet.
inline void PackPacket(Packet &packet, uint8_t type, uint8_t sequence,
FoodCondition condition,
                        uint8_t additional_data = 0)
{
    packet.header = MakeHeader(type, sequence);
    packet.payload = MakePayload(condition, additional_data);
    packet.crc = ComputeCRC(packet.header, packet.payload);
}

// Unpacks incoming packet data.
inline bool UnpackPacket(const Packet &packet, uint8_t &type, uint8_t &sequence,
FoodCondition &condition,
                        uint8_t &additional_data)
{
    if (ComputeCRC(packet.header, packet.payload) != packet.crc)
        return false;

    uint8_t version = (packet.header >> 7) & 0x01;
    if (version != PROTO_VERSION)
        return false;

    type = (packet.header >> 5) & 0x03;
    sequence = packet.header & 0x0F;

    condition = (FoodCondition)((packet.payload >> 6) & 0x03);
    additional_data = packet.payload & 0x3F;

    return true;
}

```